

AREX-2: Advancing Self-Improving Agents through Long-Horizon Reflective Tasks

AREX Team

Beijing Academy of Artificial Intelligence (BAAI)

We present **AREX-2**, an effort to advance the self-improving capability of LLM agents, which we define as the ability to iteratively refine a solution at test time. This ability rests on two complementary capabilities: *reflection*, which produces a solution better than the current one, and *long-horizon execution*, which keeps the iteration effective over many rounds. We hypothesize that both capabilities are domain-agnostic, and can therefore be learned in scenarios that are well suited for supervision. Accordingly, we synthesize long-horizon improvement trajectories from machine learning and algorithmic programming tasks, two domains that offer verifiable feedback and reward sustained iteration. Trained on this data, our agent, built on Qwen3.8-27B, achieves strong results on MLE-bench Lite (81.8) and Frontier-CS (70.7), transfers to deep research with 84.0 on BrowseComp, 52.6 on HLE, 92.2 on GAIA, and 93.8 on DeepSearchQA, and keeps improving as its budget of rounds grows. These results show that long-horizon reflective data is an effective route toward self-improving agents.

🏠 **Homepage:** <https://github.com/VectorSpaceLab/AREX-2>

🤖 **Models:** <https://huggingface.co/collections/BAAI/arex-2>

✉ **Correspondence:** chienqhj@gmail.com, zhengliu1026@gmail.com

AREX-2 benchmark results

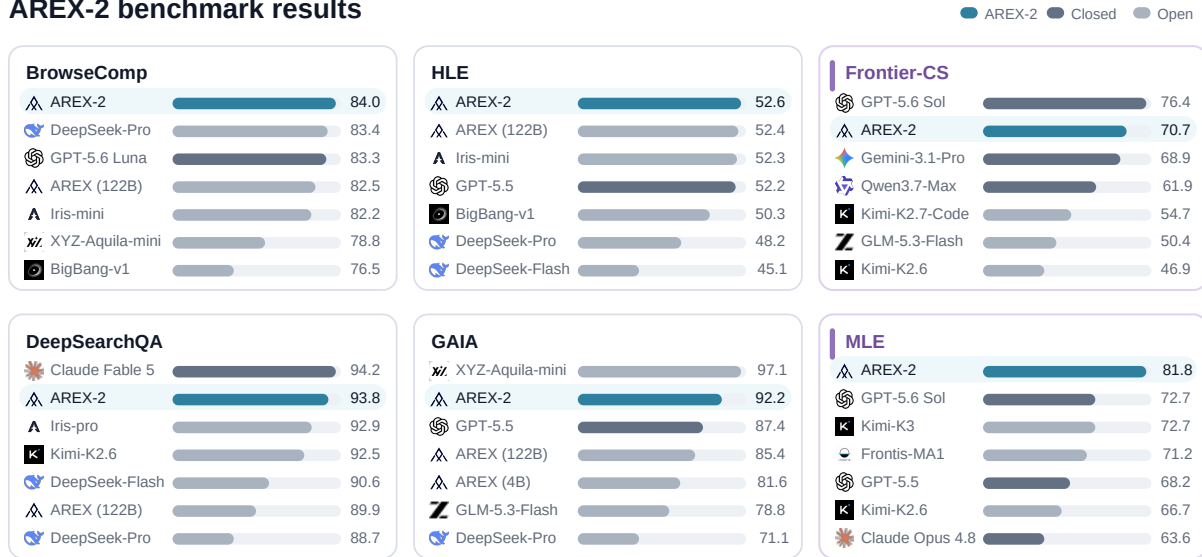


Figure 1 Results of AREX-2 on six benchmarks, compared with selected closed and open models.

1 Introduction

Hard problems are not solved in one attempt. A training pipeline, a program under a scoring function, or a research question yields to a loop of trying, measuring, and revising, and the systems that do such work well are built around that loop (Jiang et al., 2025; Novikov et al., 2025; Lu et al., 2024; Yang et al., 2026). We call one pass through the loop a *round*. In most of these systems, however, the loop is implemented largely in the scaffold rather than in the model. The harness decides when to retry and what to keep, while the model is

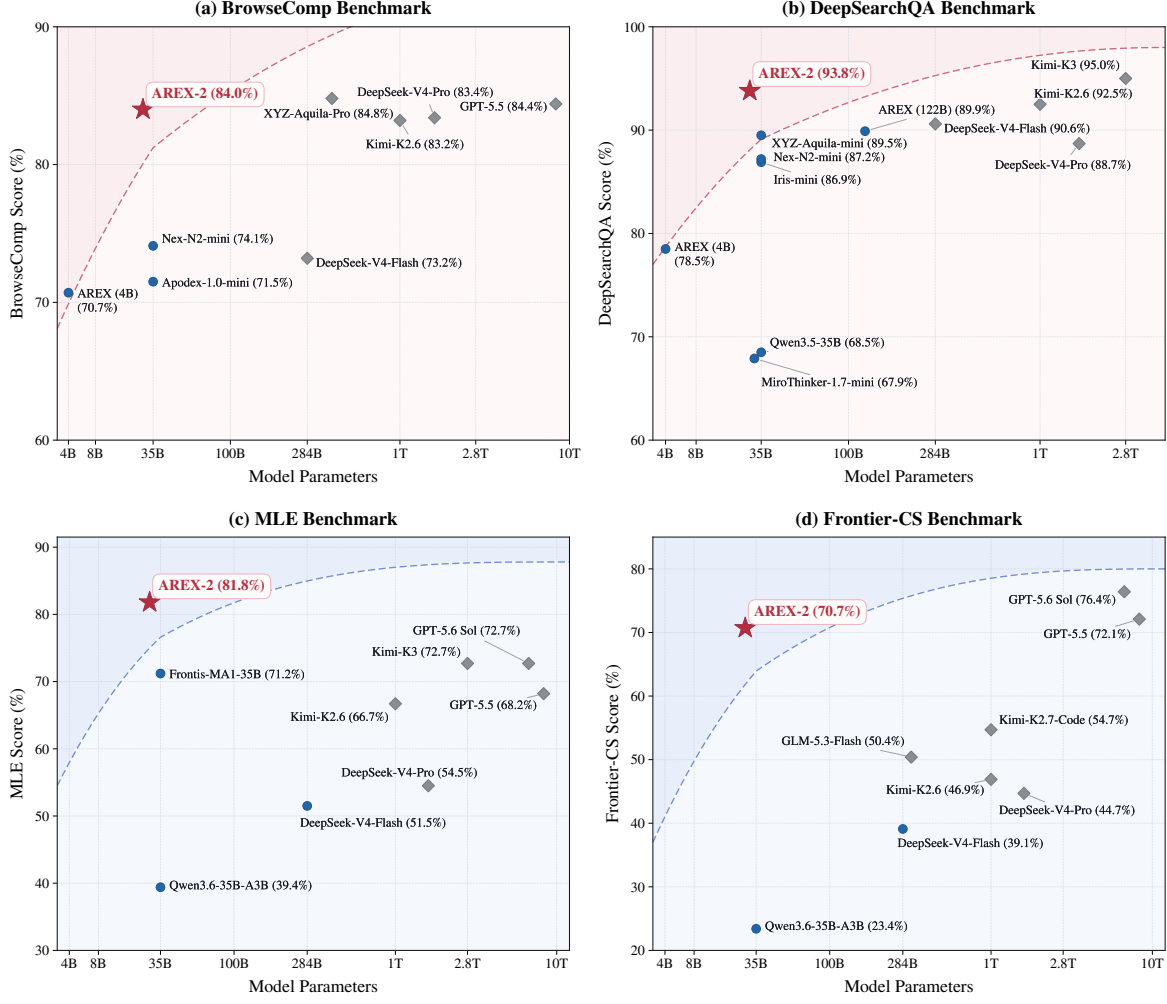


Figure 2 Model size versus performance on four benchmarks. AREX-2 (27B) matches or exceeds much larger models.

asked mainly to produce one attempt at a time. This paper asks how to move the loop inside the model, and defines *self-improvement* accordingly: the ability of an agent, given more rounds on a task, to turn them into a better solution by its own judgment of what to change. Self-improvement in this sense is test-time scaling within a single task: the more rounds the agent spends, the better its solution should become.

Whether more rounds help depends on two capabilities. *Reflection* is producing, from the current solution and what the feedback says about it, a better one. *Long-horizon execution* is sustaining this process across rounds, retaining useful findings and recovering from setbacks. Let s_t be the best score the agent has reached after t rounds and $r_t = \mathbb{E}[s_t - s_{t-1}]$ the expected gain of round t . From a fixed initial score s_0 and a budget of T rounds, the expected total improvement is

$$\mathbb{E}[s_T] - s_0 = \sum_{t=1}^T r_t. \quad (1)$$

Let $T^* \leq T$ be the number of rounds in which r_t remains non-negligible, and $\bar{r}$ the mean gain over those rounds, so that the total improvement is approximately $\bar{r}T^*$. Reflection determines $\bar{r}$, how much a productive round is worth; long-horizon execution determines T^* , how many rounds stay productive before the agent stalls. The two capabilities are therefore complementary: when $\bar{r}$ is small, further rounds add little; when T^* is short, even sharp reflection has limited cumulative effect, and a larger budget helps only up to T^* . A self-improving agent needs both, and we refer to the two together as *long-horizon reflection*.

To move the loop inside the model, both capabilities have to be learned in training, and a model learns mainly

what its training data demonstrates. Current training data for agents demonstrates little of either. It is typically built one attempt at a time: a task is posed, the model produces a solution, a verifier marks it pass or fail, and the passing solutions are kept for training (Pan et al., 2025; Yang et al., 2025; Jain et al., 2025). This follows the general practice of fine-tuning a model on its own successful outputs (Zelikman et al., 2022; Gulcehre et al., 2023; Yuan et al., 2023). Data built this way shows a model what a correct solution looks like, but not how a solution is improved. The intermediate attempts, the feedback they received, and the revisions that followed are all discarded, and because solutions that pass immediately are the easiest to collect, long processes of improvement are the first to be lost. As a result, trajectories in which an agent works on one problem for hours, measuring and revising until it ends far above where it began, are almost absent from training data. Such trajectories have to be constructed, which raises two questions: in which domains to construct them, and how.

For the first question, we start from a hypothesis: **long-horizon reflection is a meta-skill that is not tied to any one domain**. Judging where a solution falls short, deciding what to change, and continuing after a failed round are the same acts whether the solution is a training pipeline, a program, or a research report. If the hypothesis holds, the training domain need not resemble the target domain. It should instead be chosen for how well it allows the meta-skill to be supervised, and what is learned there should transfer to other domains. By this criterion we choose two domains, machine learning engineering and algorithmic programming, for three reasons. First, their feedback is unambiguous: a validation score or a judge’s verdict shows directly whether a revision helped. Second, they leave room for sustained improvement: a solution can keep getting better over many rounds, from a baseline to a tuned ensemble or from a brute-force program to a near-optimal one, so long trajectories are rewarded (Chan et al., 2025; Mang et al., 2025). Third, their source material is abundant: GitHub and online judges offer a large supply of problems, code, and tests. Together, these properties make the two domains well suited to supervising self-improvement.

For the second question, we construct the data in three steps. First, a teacher model turns source material from GitHub and online judges into executable environments, each consisting of a task and a scoring function. Second, an agent works on each task over many rounds, revising its solution according to the feedback it receives. Third, we select trajectories as a whole: a trajectory is kept if its final score is high and its process satisfies the task’s requirements, and no individual round is required to succeed. This choice is deliberate. A trajectory selected by its final outcome still contains failed runs, regressions, and abandoned approaches, which filtering at the level of individual steps would remove. Keeping them teaches the model to continue iterating after a setback. Removing them would leave the model with only examples in which every step succeeded, and with nothing to learn from when a round fails.

We train **AREX-2** from Qwen3.8-27B (Qwen Team, 2026) on this data, together with the deep-research data of the AREX recipe (Lu et al., 2026). In the two training domains, it reaches 81.8 on MLE-bench Lite, the highest score among the systems we compare, and 70.7 on Frontier-CS, the highest among open-weight models (Figure 1). In deep research, for which we added no new training data, it still improves over the previous AREX models, reaching 84.0 on BrowseComp, 52.6 on HLE, 92.2 on GAIA, and 93.8 on DeepSearchQA: long-horizon reflection carries over beyond the domains it was learned in. With only 27B parameters, AREX-2 compares favorably with substantially larger models (Figure 2), and it keeps improving as its budget of rounds grows.

2 Method

2.1 Self-Improvement as Test-Time Scaling

An environment is a pair $e = (\sigma, S)$, where σ is the task given to the agent and S is a scoring function that maps a candidate solution y to a scalar $S(y)$. Examples of S are the validation metric of a trained model and the fraction of hidden tests a program passes; we orient S so that higher is better. The agent works in e over rounds. In round t it submits a solution y_t and receives feedback f_t , which consists of the score and whatever else the environment reports, such as logs, errors, and timings. A round includes everything the agent does before the submission: reading, editing, running, and searching. A trajectory of n rounds is

$$\tau = (y_0, f_0, y_1, f_1, \dots, y_n, f_n), \quad s_t = \max_{k \leq t} S(y_k), \quad (2)$$

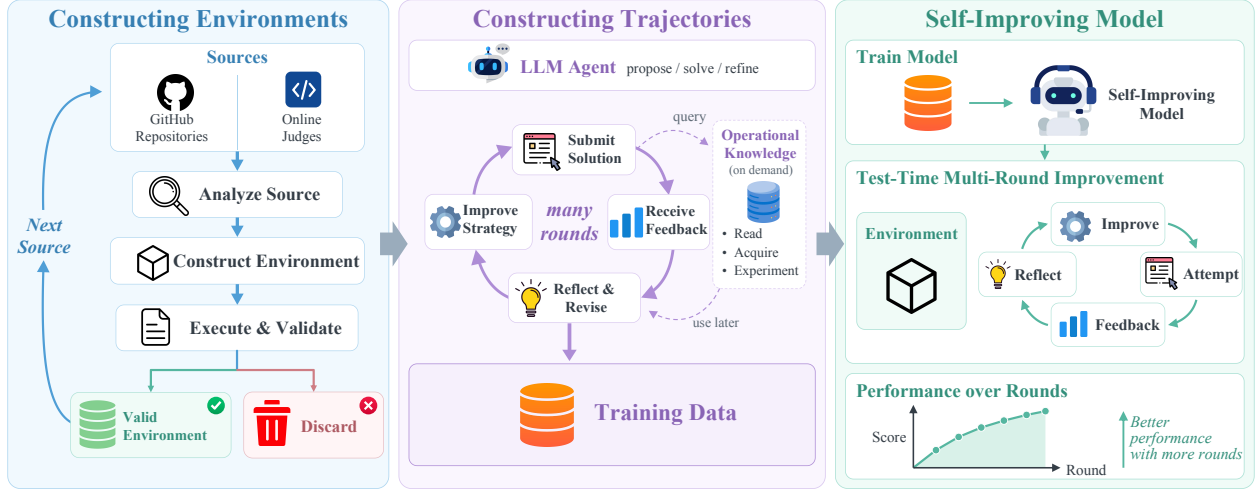


Figure 3 Overview of AREX-2. **Left:** environments are constructed from GitHub repositories and online judges, and are kept only if they pass execution and validation. **Middle:** an agent improves its solution over many rounds, acquiring operational knowledge on demand, and its trajectories become training data. **Right:** the model trained on this data improves its solution over multiple rounds at test time.

where s_t is the best score reached after t rounds. Under a policy π_θ , the expected gain of round t is

$$r_t = \mathbb{E}_{\pi_\theta}[s_t - s_{t-1}]. \quad (3)$$

For a fixed initial score s_0 and a budget of T rounds, the expected total improvement is therefore

$$\mathbb{E}_{\pi_\theta}[s_T] - s_0 = \sum_{t=1}^T r_t, \quad (4)$$

as in Equation (1). The budget T is the resource scaled at test time. It is given to the agent, not learned by it; what the policy determines is how much of the budget it can use productively. For a small threshold ϵ , let

$$T^* = |\{t \leq T : r_t > \epsilon\}|, \quad \bar{r} = \frac{1}{T^*} \sum_{t \leq T^*} r_t \quad (5)$$

be the number of rounds that remain productive and the mean gain over those rounds. The total improvement is then $\bar{r}T^*$, up to an error of at most ϵT . Reflection determines $\bar{r}$ and long-horizon execution determines T^* , so a larger budget helps only up to T^* . Because s_t is the best score so far, a single submission may score below it without lowering s_t , and r_t decreases as the score approaches its ceiling.

An agent is self-improving in e to the degree that more rounds lead to better solutions in expectation. Our hypothesis is that long-horizon reflection is a meta-skill: the behaviors that produce such improvement, which are using feedback, acquiring operational knowledge, and recovering from setbacks, transfer across families of environments even when their tasks and scoring functions differ.

We train π_θ by imitation, on trajectories that exhibit these behaviors. The rest of this section describes how the trajectories are obtained: how environments are built (Section 2.2), how an agent is run in them to produce trajectories of sustained improvement (Section 2.3), and how the trajectories are selected and turned into supervision (Section 2.4).

2.2 Constructing Environments

We build environments from two kinds of source: GitHub repositories, which supply machine learning tasks, and online judges, which supply algorithmic programming tasks. A source is not yet an environment. A repository has code, data, and a way of measuring a model, but it does not state a task. A judge problem

has a statement and tests, but it has no sandbox in which an agent can iterate. A teacher model therefore turns each source into an environment (σ, S) .

For a repository, the teacher reads the code and identifies the metric the repository measures. It then writes a task that asks the agent to improve this metric, a scoring script that computes the metric on a held-out split the agent cannot access, and a sandbox with the repository installed and its data in place. For a judge problem, the task is the problem statement, the sandbox contains a compiler and the sample cases, and S is the score assigned by the hidden tests. Wherever the judge allows it, this score is graded, not binary: for example, the fraction of tests passed, or the quality of a heuristic solution relative to the best known one. A graded score makes each round informative, whereas a binary verdict says only that the agent has not succeeded.

An environment is admitted if it satisfies two conditions, both checked by execution. First, the reference solution that comes with the source, which is the repository’s own model or the judge’s accepted submission, must run under S and obtain a score. This shows that the environment works. Second, a simple baseline, which is the repository as found or a straightforward first program, must score well below the reference. This shows that the environment leaves room for improvement. Environments that fail the second condition are dropped, because they cannot produce trajectories of sustained improvement.

2.3 Constructing Trajectories

A trajectory is produced by running a teacher agent in an environment, under three conditions designed to elicit long-horizon reflection.

More rounds. The agent is given a budget of rounds and wall-clock time far larger than a single attempt needs, on the order of hours and hundreds of tool calls per task, and it is told what the budget is. Knowing the budget changes how the agent works. It can establish a baseline first, use early rounds to measure, and use later rounds for the revisions that the measurements suggest, instead of submitting its best guess at once. The trajectory thus demonstrates how to turn a larger round budget into cumulative improvement, and the round budget is the same resource that we scale at test time.

Operational knowledge. To improve a solution, the agent has to know how the environment works: which API the repository exposes, what its data loader expects, which optimizations the constraints of a problem permit. We call this *operational knowledge*. It is not given in the task statement, so the agent acquires it while working, by reading the repository’s source and documentation, searching for the papers and libraries that the code depends on, and running small experiments in the environment. The agent then uses what it has learned in the rounds that follow. All of these actions remain in the trajectory, so a model trained on the trajectory learns to investigate how an environment works as part of improving a solution. Part of this knowledge can be provided as *skills*, which are compact documents placed in the agent’s context. General skills are prepared in advance from machine learning repositories and common practices. Task-specific skills are built by searching for information related to a task, and material about the task itself is excluded. We provide skills to the agent that produces the trajectories, because they make it capable enough to produce trajectories worth imitating. The trained model learns from these trajectories how to use such knowledge, and skills remain available to it at test time.

Feedback in the loop. Each round ends with a submission, and the agent sees the score and the environment’s report before it decides what to do next. This is the point at which reflection takes place: the agent compares the result with what it expected, keeps or reverts its change, and forms its next hypothesis. Rounds that lower the score are not removed from the trajectory. Each is followed by the agent’s response to it, and this response is what we want the model to learn.

For each environment, the result is a set of trajectories in which a capable agent works on one task over many rounds, with its reasoning, searches, failures, and recoveries all recorded.

2.4 Training

Trajectory selection. A trajectory is kept or discarded as a whole, according to two conditions. First, its final score must be high: the best submission must reach a threshold set relative to the reference, so that the trajectory demonstrates a real improvement. Second, its process must satisfy the task’s requirements: the transcript follows the user-assistant-tool format, every tool call has an observation, the run terminates in a well-formed state, and the agent has not violated the rules of the task, for example by reconstructing answers from the held-out split or exposing credentials. We place no condition on individual rounds. A kept trajectory may therefore contain failed runs, regressions, and abandoned approaches. We keep them deliberately, because they show the model how to continue after a setback.

Supervision within a trajectory. The whole trajectory is kept as context, and the loss is applied to the decisions that move the solution forward. We group the agent’s outputs into steps, where a step is one decision together with the actions issued before the next observation. Failed attempts and regressions stay in the context, so that the model sees the state from which it has to recover. The loss is applied to the decisions that follow them and make progress: diagnosing the failure, repairing it, changing strategy, running the next experiment, and submitting an improved solution.

Steps that make no progress receive no loss. These include repeated polling, calls that return no new observation, and near-duplicate turns. System messages, environment reports, and retrieved documents also receive no loss, since they condition the model but are not outputs to imitate. The model thus learns how to respond to a failure and turn it into a later improvement, while the failure itself stays in the context.

Data and model. We combine the selected trajectories with the deep-research data of the AREX recipe (Lu et al., 2026), which we leave unchanged, and fine-tune Qwen3.8-27B (Qwen Team, 2026) on the mixture to obtain AREX-2. The new trajectories are the only difference from the previous recipe. The cross-domain results in Section 3 can therefore be read as transfer: any gain of AREX-2 on deep research over models trained with the previous recipe does not come from new deep-research data.

3 Experiments

Our experiments ask whether long-horizon improvement trajectories produce strong performance in their source domains, whether the learned capabilities extend to deep research, and whether the trained agent turns additional test-time computation into continued progress. We first report overall benchmark results, then examine how the agent improves during execution and how the data-generation recipe teaches that behavior.

3.1 Experimental Setup

Benchmarks. We evaluate AREX-2 on six benchmarks covering four capabilities: algorithmic programming, machine learning engineering, deep research, and general agentic reasoning. **Frontier-CS** (Mang et al., 2025) evaluates algorithmic programming on open-ended problems with graded scores, where an agent improves its solution through repeated implementation, testing, and submission. **MLE-bench Lite** (Chan et al., 2025) (MLE-Lite for short) measures machine learning engineering capabilities, where agents must complete end-to-end ML workflows involving data analysis, experimentation, implementation, and model evaluation. For deep research and agentic reasoning, we include **BrowseComp** (Wei et al., 2025), **HLE** (Phan et al., 2025), **GAIA** (Mialon et al., 2023), and **DeepSearchQA** (Gupta et al., 2026). These benchmarks cover deep research, tool-augmented reasoning, information gathering, and multi-step task completion. Following standard evaluation protocols, we report F1 for **DeepSearchQA**, Any Medal for **MLE-Lite**, and accuracy for the remaining benchmarks. Any Medal is the percentage of competitions in which the agent earns a medal, averaged over three seeds.

Evaluation protocols. For non-coding benchmarks, we follow the evaluation protocol of **AREX** (Lu et al., 2026), setting the maximum number of inner turns to 300 and the overall maximum number of turns to 1500. For coding-related evaluations, we adopt the official agent-based evaluation settings of each benchmark. Specifically, **Frontier-CS** is evaluated under the Agent Track with a maximum execution budget of 5 hours per task, while **MLE-Lite** follows the OpenMLE evaluation protocol (Yang et al., 2026) with a maximum budget of

Model	Size	Frontier-CS	MLE-Lite
<i>Closed-Weight Models</i>			
GPT-5.6 Sol	–	76.4	72.7
Claude Opus 4.8	–	74.5	63.6
GPT-5.5	–	72.1	68.2
Gemini-3.1-Pro	–	68.9	–
Qwen3.7-Max	–	61.9	–
<i>Open-Weight Models</i>			
Kimi-K3	2.8T	–	72.7
Naive-N0.5-Flash	309B	–	73.7
DeepSeek-V4-Pro	1.6T	44.7*	54.5
DeepSeek-V4-Flash	284B	39.1*	51.5
Kimi-K2.7-Code	1T	54.7*	–
GLM-5.3-Flash	320B	50.4*	–
Kimi-K2.6	1T	46.9	66.7
Frontis-MA1-35B	35B	–	71.2
BigBang-V1	35B	–	59.1
Qwen3.6-35B-A3B	35B	23.4*	39.4
AREX-2	27B	70.7	81.8

Table 1 Comparison on coding and machine learning engineering benchmarks. Model size refers to the total number of parameters. Frontier-CS results are reported on the 188-task Agent Track. * denotes results reproduced by us. MLE-Lite results are reproduced and reported by OpenMLE. MLE-Lite reports Any Medal (mean over three seeds) on MLE-bench Lite. AREX-2 is evaluated on MLE-Lite with skills in its context (Section 3.3.3).

12 hours per task. On MLE-Lite, AREX-2 is evaluated with skills in its context; Section 3.3.3 reports how much the skills and the training each contribute. All evaluations are conducted using the corresponding benchmark environments, allowing agents to iteratively reason, act, and refine their solutions within the budget.

3.2 Overall Results

Learning to solve verifiable tasks. Table 1 evaluates performance in the domains used to construct the new training trajectories. AREX-2 reaches an Any Medal rate of 81.8 on MLE-bench Lite, the highest score in the table: 8.1 points above the strongest baseline, Naive-N0.5-Flash, and 9.1 points above the strongest closed-weight baseline, GPT-5.6 Sol. On Frontier-CS, it achieves 70.7, exceeding the strongest reported open-weight baseline by 16.0 points and placing within 5.7 points of the strongest closed-weight system. At 27B parameters, AREX-2 therefore combines leading performance on machine learning engineering with competitive performance on algorithmic programming among the systems compared here.

Extending capabilities without new research data. Table 2 evaluates cross-domain transfer: no newly constructed trajectory is a search task, while the deep-research training data remains unchanged from the previous AREX recipe. AREX-2 obtains 84.0 on BrowseComp, 52.6 on text-only HLE, 92.2 on GAIA, and 93.8 on DeepSearchQA. It leads the reported models with at most 40B parameters on BrowseComp, text-only HLE, and DeepSearchQA; on GAIA, it ranks behind XYZ-Aquila-mini and Agents-A1 but above the remaining small models with reported results. Its performance is also competitive with substantially larger systems: it exceeds DeepSeek-V4-Pro and Kimi-K2.6 on BrowseComp and ranks third on DeepSearchQA among the models in the table. Most importantly, AREX-2 surpasses both models trained with the previous recipe, AREX (4B) and AREX (122B), on all four benchmarks.

Overall. AREX-2 combines strong performance in machine learning engineering and algorithmic programming with substantial gains in deep research over the previous AREX models, although it uses the same deep-research training data. This pattern is consistent with the hypothesis in Section 1 that long-horizon

Model	Size	BrowseComp	HLE	GAIA	DeepSearchQA
<i>Frontier Models</i>					
GPT-5.6 Sol	–	90.4	58.0*	–	–
GPT-5.6 Terra	–	87.5	–	–	–
GPT-5.6 Luna	–	83.3	–	–	–
Kimi-K3	2.8T	91.2	56.0*	–	95.0
Claude Fable 5	–	88.0	64.5*	–	94.2
Claude Opus 4.8	–	84.3	57.9*	–	93.1
GPT-5.5	–	84.4	52.2*	87.4	–
Gemini-3.1-Pro	–	85.9	51.4*	80.6	93.3
<i>Large Models (>40B)</i>					
GLM-5	744B	75.9	50.4	70.0	–
Kimi-K2.6	1T	83.2	54.0*	80.6	92.5
GLM-5.3-Flash	320B	–	55.3*	78.8	–
DeepSeek-V4-Flash	284B	73.2	45.1	57.5	90.6
DeepSeek-V4-Pro	1.6T	83.4	48.2	71.1	88.7
MiroThinker-1.7	235B	74.0	42.9	82.7	72.1
XYZ-Aquila-pro	397B	84.8	53.3	–	92.5
Iris-pro	397B	88.6	56.4	–	92.9
AREX (122B)	122B	82.5	52.4	85.4	89.9
<i>Small Models ($\leq 40B$)</i>					
Tongyi-DeepResearch-30B	30B	43.4	32.9	70.9	–
Qwen3.5-35B	35B	61.0	47.4	80.0	68.5
XYZ-Aquila-mini	35B	78.8	51.1	97.1	89.5
BigBang-V1	35B	76.5	50.3	–	–
Quest-35B	35B	64.6	37.2	80.8	–
Apodex-1.0-mini	35B	71.5	46.8	–	82.2
Agents-A1	35B	75.5	47.6	96.0	–
MiroThinker-1.7-mini	30B	67.9	36.4	80.3	67.9
Iris-mini	35B	82.2	52.3	–	86.9
AREX (4B)	4B	70.7	40.6	81.6	78.5
AREX-2	27B	84.0	52.6	92.2	93.8

Table 2 Comparison on general agentic reasoning and deep research benchmarks. * denotes results reported on the full HLE set; unmarked results use the text-only subset.

reflection is a meta-skill: what the model learns from the two training domains carries over beyond them. A gap nevertheless remains on the strongest BrowseComp and HLE results.

3.3 Detailed Analysis

The results in Section 3.2 show what AREX-2 achieves, but not how. This section examines two questions that follow from the formalization in Section 2.1. The first is whether AREX-2 turns a larger budget into a better solution, that is, whether its effective horizon T^* is long. We test this in two settings: one in which the agent receives a score after each submission, and one in which it receives no feedback on correctness. The second question is how much each part of our approach contributes: operational knowledge, training, and a larger budget. We test this with a stage-wise ablation.

3.3.1 Round Scaling with Feedback

Setup. We use the 188 problems of the Frontier-CS Agent Track, with a budget of five hours per problem, measured from the start of the agent session. The agent works on each problem in rounds. In a round, it

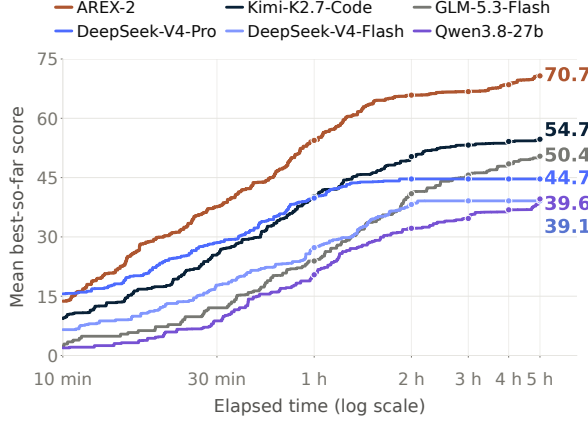


Figure 4 Test-time scaling for agentic coding tasks. Mean best-so-far score across 188 Frontier-CS Agent Track problems. Elapsed time starts at each task’s agent session.

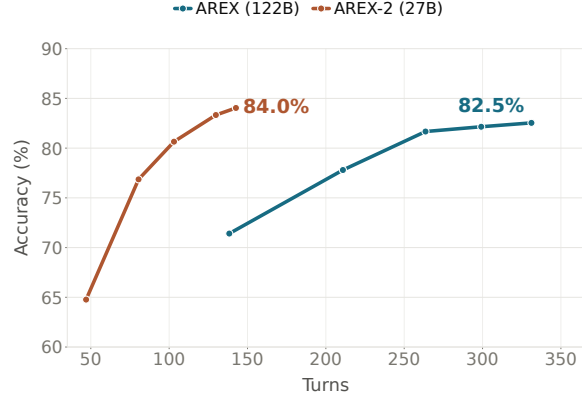


Figure 5 Test-time scaling for capability transfer on BrowseComp, showing accuracy versus average turns with no correctness feedback provided during evaluation.

implements or revises a solution, compiles it, runs the provided sample cases, and writes additional randomized and edge-case tests of its own. It then submits the solution and receives a score with evaluation feedback, which it uses to decide what to change in the next round. AREX-2 runs with our optimized harness. Figure 4 reports the mean best-so-far score over time.

Results. AREX-2 improves throughout the five hours. It does not start with the highest score, but it takes the lead as the budget grows, reaching 54.4 after one hour, 65.9 after two hours, and 70.7 after five hours. In contrast, DeepSeek-V4-Pro stops improving at 44.7 after two hours, and DeepSeek-V4-Flash at 39.1 after three hours. For AREX-2 the later hours remain productive: it gains 4.8 points between the second and the fifth hour, of which 2.2 points come in the final hour.

What it verifies. The advantage of AREX-2 comes from sustained improvement, and not only from a strong start. Its gains become smaller over time but remain positive until the end of the budget, which corresponds to a long effective horizon T^* . The two baselines reach their effective horizon within two to three hours, after which a larger budget no longer helps them.

3.3.2 Round Scaling without Feedback

Setup. We use BrowseComp, a deep-research benchmark for which no new trajectories were added in training. Unlike in the previous setting, the agent receives no score during a run, so it never learns whether its current answer is correct. AREX-2 follows a recursive research process with two loops. The inner loop gathers evidence and produces a provisional answer. The outer loop decides whether to accept the answer, refine it, or restart, based on the model’s own confidence and the state of the trajectory. We vary the budget, measured as the average number of turns per question, where a turn is one LLM call, and report accuracy in Figure 5. For comparison, we run AREX (122B), trained with the previous recipe, under the same process.

Results. The accuracy of AREX-2 rises steadily with the budget, from 64.8 at 47 turns to 84.0 at 143 turns, with diminishing returns as the budget grows. AREX (122B) also improves with more turns, but far less efficiently. At a similar budget of about 140 turns, AREX-2 is more than 12 points ahead. It also exceeds the final accuracy of AREX with less than half of the turns, and its gain per turn is about three times as large. AREX-2 achieves this with 27B parameters, less than a quarter of the size of AREX.

What it verifies. AREX-2 can improve its answer without an external correctness signal. The gains come from the model continuing to search, reassess, and refine by its own judgment, and not from correcting an answer after being told that it was wrong. Its gain per turn is also about three times that of AREX,

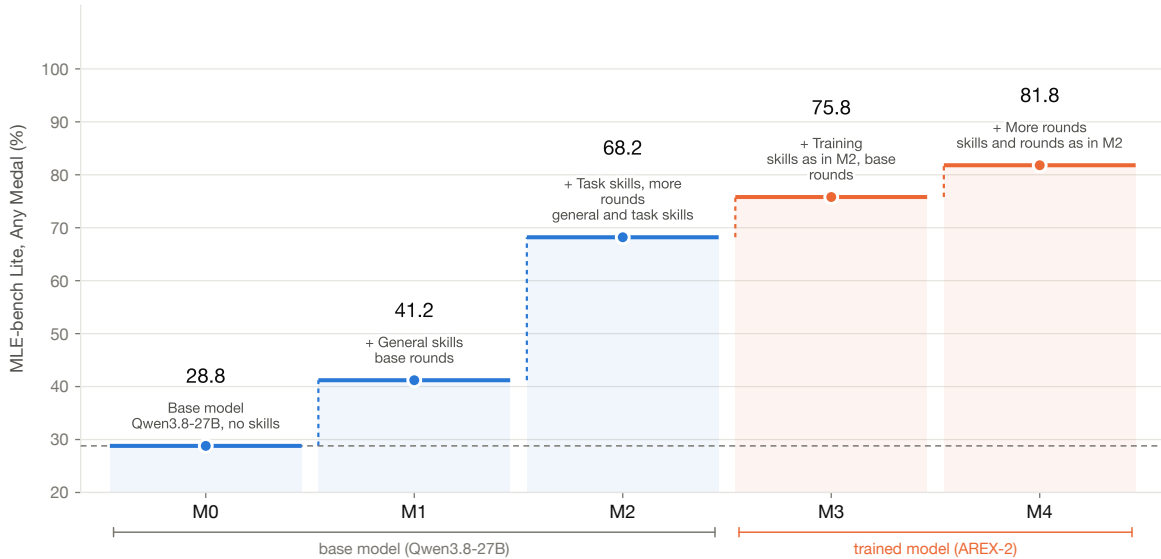


Figure 6 Stage-wise ablation on MLE-bench Lite. Stages M0 to M2 use the base model, and M3 and M4 use AREX-2.

which corresponds to a larger $\bar{r}$ in Section 2.1. Since the two models share the same deep-research training data and the new trajectories contain no research tasks, this difference is evidence that long-horizon reflection transfers across domains.

3.3.3 Stage-wise Ablation

Setup. We measure how much each part of our approach contributes, in five stages that are all evaluated on MLE-bench Lite (Figure 6). The stages vary three factors: the model, the skills in its context, and the number of rounds. Stages M0 to M2 use the base model, Qwen3.8-27B. M0 has no skills, M1 adds general skills, and M2 adds task-specific skills and more rounds. Stages M3 and M4 use the trained model, AREX-2, with the same skills as M2. M3 uses the same number of rounds as M0 and M1, and M4 uses the same number as M2. General skills are prepared in advance from machine learning repositories and common practices. Task-specific skills are built by searching for information related to a task, and material about the task itself is excluded.

Results. The base model scores 28.8. General skills raise the score to 41.2, and task-specific skills with more rounds raise it to 68.2. With the same skills, AREX-2 reaches 75.8 with the base number of rounds and 81.8 with more rounds.

What it verifies. The ablation supports three observations. First, operational knowledge matters: skills and more rounds raise the base model from 28.8 to 68.2 without any training. Second, training adds to what skills provide. M2 and M4 have the same skills and the same number of rounds and differ only in the model, and AREX-2 exceeds the base model by 13.6 points. AREX-2 is ahead even with fewer rounds: M3 exceeds M2 by 7.6 points. Third, the trained model turns more rounds into further improvement, gaining 6.0 points from M3 to M4, which is consistent with the scaling results above.

4 Conclusion

We set out to move the improvement loop inside the model. We defined self-improvement as turning more rounds on a task into a better solution, and traced it to two complementary capabilities: reflection raises the gain per round, and long-horizon execution keeps the rounds productive. Neither is learned from finished solutions, so we built data that records the work: trajectories in which an agent improves a solution over many rounds, in environments compiled from machine learning repositories and algorithmic problems.

Trained on them, AREX-2 achieves leading results on MLE-bench Lite and Frontier-CS, and improves on deep research as well, for which no new data was added. It keeps improving as its budget grows: on Frontier-CS through the fifth hour, and on BrowseComp without any correctness signal. A stage-wise ablation further shows that operational knowledge, training, and a larger budget each contribute to the result. Long-horizon reflection, learned where it can be supervised, carries over beyond the domains it was learned in. The path forward is to widen the training domains, lengthen the horizons, and close the loop by letting the model’s own trajectories become its next training data.

Contributions

Core Contributors: Hongjin Qian¹, Chaofan Li¹, Kun Luo¹, Wenqing Wei¹, Jianlyu Chen¹, Zheng Liu²

Participants: Shuqi Lu, Yuyang Hu, Hongwang Xiao, Hui Wang, Chaozhao Li, Qiwei Ye

Advisors: Zhicheng Dou, Defu Lian

References

- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.
- Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced Self-Training (ReST) for language modeling. *arXiv:2308.08998*, 2023.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. DeepSearchQA: bridging the comprehensiveness gap for deep research agents. *arXiv:2601.20975*, 2026.
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: procedural environments and hybrid verifiers for scaling Open-Weights SWE agents. *arXiv:2504.07164*, 2025.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-Driven exploration in the space of code. *arXiv:2502.13138*, 2025.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated Open-Ended scientific discovery. *arXiv:2408.06292*, 2024.
- Shuqi Lu, Chaofan Li, Kun Luo, Zhang Zhang, Hui Wang, Hongwang Xiao, Lei Xiong, Jiahao Wang, Sen Wang, Xiyan Jiang, et al. AREX: towards a recursively Self-Improving agent for deep research. *arXiv:2607.21461*, 2026.
- Qiuyang Mang, Wenhao Chai, Zhifei Li, Huanzhi Mao, Shang Zhou, Alexander Du, Hanchen Li, Shu Liu, Edwin Chen, Yichuan Wang, et al. FrontierCS: evolving challenges for evolving intelligence. *arXiv:2512.15699*, 2025.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. *arXiv:2311.12983*, 2023.
- Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: a coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*, 2025.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *International Conference on Machine Learning*, 2025.
- Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam. *arXiv:2501.14249*, 2025.
- Qwen Team. Qwen3.8. <https://github.com/QwenLM/Qwen3.8>, 2026.

¹These authors contributed equally to this work.

²Zheng Liu is the project leader.

- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: a simple yet challenging benchmark for browsing agents. *arXiv:2504.12516*, 2025.
- John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: scaling data for software engineering agents. *arXiv:2504.21798*, 2025.
- Junlin Yang, Che Jiang, Yu Fu, Tianwei Luo, Can Ren, Weizhi Wang, Kaikai Zhao, Hongyi Liu, Yuxin Zuo, Yuru Wang, et al. Frontis-MA1: training an AI4AI model towards recursive Self-Improvement in machine learning engineering. *arXiv:2607.28568*, 2026.
- Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv:2308.01825*, 2023.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.